Tenant-Aware Authorization

what can you do, and in which tenant?

Watch the 4-min video

In one line: take the tenant from a **signed token**, set it once per request, enforce it automatically in the data layer with **row-level security** as a backstop, carry it to every downstream call, queue and job, and give support staff a separate, audited break-glass path.

1 Invoice 124

A support engineer at Acme changes `/invoices/123` to `/invoices/124` and sees Globex's invoice. Login worked. The role check worked: she may read invoices. Nobody asked **whose** invoices. This is **Broken Object Level Authorization** (BOLA, also called IDOR), number one in the OWASP API Security Top 10 (2023). In a multi-tenant system every authorization decision has two parts: what may you do, and in which tenant.



the bug in the hook: steps 1 and 2 passed, step 3 was never asked

Figure 1. Authorization in SaaS is three checks. The tenant check is the one that is easy to forget, so it must not depend on remembering.

Hard-to-guess IDs (UUIDs instead of 123, 124) make enumeration harder but fix nothing: IDs leak through URLs, logs, emails and exports. The fix is a tenant check on every object access, enforced by structure rather than discipline.

2 The tenant comes from the token

At sign-in the identity provider issues a JWT with the user ID, roles and a tenant claim (Microsoft Entra ID, for example, puts the directory in `tid`; with your own IdP you add a custom claim). The server verifies the **signature** against the issuer's published keys (JWKS), pins the allowed algorithms (never accept `alg: none`), and checks `iss`, `aud` and `exp`. Because the token is signed, the client cannot change its tenant. Never trust a tenant ID from a header, query string or body.

Users who belong to several tenants (an accountant serving five clients) are the exception. Let them pick a tenant in the URL or a header, but check it on the server against their **memberships**, or better, exchange the login token for a short-lived, tenant-scoped token (OAuth 2.0 Token Exchange, RFC 8693) so everything downstream sees one verified tenant.

3 Set the context once, enforce it below

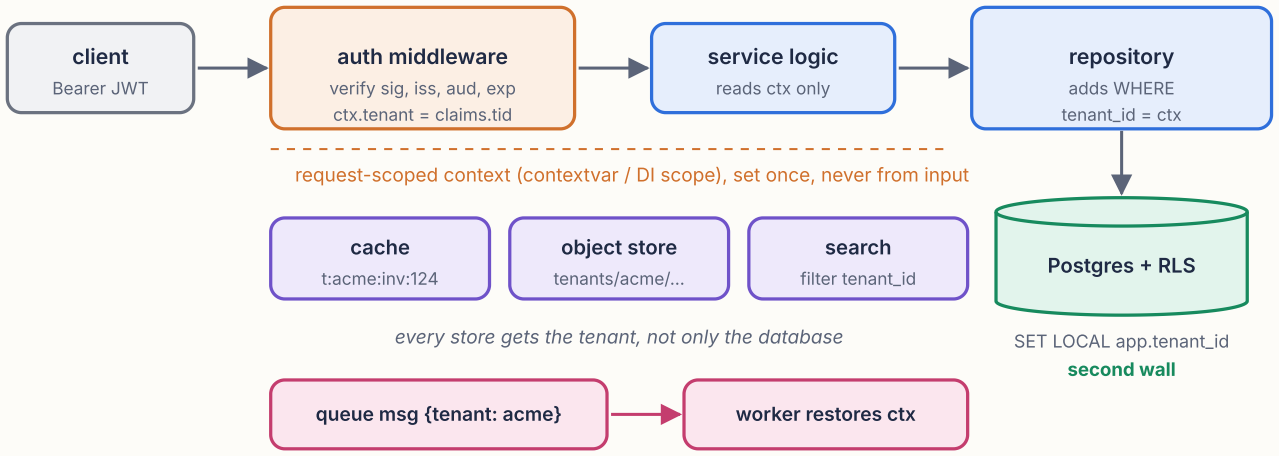


Figure 2. Tenant context is established at the edge and read by every layer. Each store also partitions by tenant.

A middleware or dependency reads the verified claims and stores the tenant in a **request-scoped context** (a Python `contextvar`, a scoped DI service in .NET, a request attribute in Spring). Services and repositories read it from there; nobody passes it by hand, so nobody can forget it.

- **Data layer, automatically.** The repository adds the tenant filter to every query, or the ORM does it globally: EF Core global query filters (`HasQueryFilter`), Hibernate 6 `@TenantId`. Writing a query without the filter should require an explicit, reviewable opt-out.
- **Database backstop.** Postgres RLS returns only rows matching `current_setting('app.tenant_id')`, set with `SET LOCAL` in each transaction so a pooled connection never carries one tenant's setting into another's request. Connect as a role that is not the table owner (lesson 42).
- **Every other store.** Cache keys prefixed `t:{tenant}:`; object storage under `tenants/{tenant}/` with credentials or pre-signed URLs scoped to that prefix; a tenant filter on every search query (or an index per tenant); tenant in every analytics row.
- **Answer 404, not 403**, for another tenant's object, so the API doesn't confirm that it exists.

4 Context must travel

A request that calls another service passes the user's token on, or exchanges it for a downstream token that keeps the tenant claim. Internal calls between your own services can use a platform-signed header, never an unsigned one that any caller could set. A message on a queue carries the tenant in its envelope; the worker **restores the context before touching data**. Fail closed: a job or query that runs with no tenant must raise an error, not run unfiltered. Scheduled jobs that truly span tenants (billing runs, cleanup) loop over tenants explicitly, setting the context for each.

Test it like a feature: an automated suite creates two tenants and calls every endpoint with tenant A's token and tenant B's object IDs; every call must return 404. Run it in CI so new endpoints can't skip it.

5 Break-glass access for your own staff

Your support engineers and operators sometimes need to see a customer's data. Treat that as a separate, exceptional path, never as an ordinary tenant role: a distinct platform role, a reason and ticket, often an approval (by a second engineer, or by the customer, as with Microsoft's Customer Lockbox), access limited in time and to one tenant, and every action written to the tenant-visible audit trail (lesson 48).

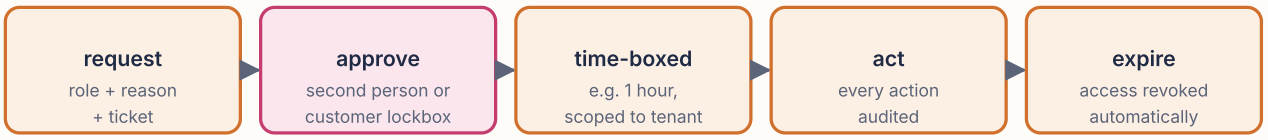


Figure 3. Break-glass access: requested, approved, time-boxed, audited, and revoked automatically.

6 Worked example: a FastAPI tenant dependency

```
async def current_tenant(token: str = Depends(oauth2_scheme)) -> str:
    key = jwks.get_signing_key_from_jwt(token).key
    claims = jwt.decode(token, key, algorithms=["RS256"],   # never 'none'
                        audience="api://invoices", issuer=ISSUER)  # checks exp
    return claims["tid"]

@app.get("/invoices/{inv_id}")
async def get_invoice(inv_id: UUID, tenant: str = Depends(current_tenant)):
    async with tenant_session(tenant) as db:      # SET LOCAL app.tenant_id
        inv = await repo.invoice(db, inv_id, tenant)  # AND tenant_id = :tenant
    if inv is None:
        raise HTTPException(status_code=404)      # not 403: don't confirm it exists
    return inv
```

Figure 4. PyJWT verifies signature, audience, issuer and expiry; the route can't run without a tenant, and the repository requires it.

WORKED EXAMPLE: ACME ASKS FOR GLOBEX'S INVOICE

The `current_tenant` dependency verifies the JWT and returns `acme`. Every tenant route declares it, and a test fails any route that doesn't. `tenant_session` opens a transaction and runs `SET LOCAL app.tenant_id = 'acme'`; the repository queries `WHERE id = :id AND tenant_id = :tenant`. Invoice 124 belongs to Globex, so the query finds nothing and the API answers 404. If a future developer forgets the filter in a new query, RLS still returns nothing, and the cross-tenant test suite fails the build.

COMMON MISTAKES

- Reading the tenant from an `X-Tenant-ID` header or request body.
- Filtering by tenant in list endpoints but not in get-by-ID, update and delete.
- Background jobs that run without a tenant and therefore see everything.
- Giving support staff a normal admin role inside customer tenants instead of audited break-glass access.

7 Key terms

BOLA / IDOR: accessing another object by changing its ID

Tenant context: request-scoped tenant set once at the edge

Token exchange: swap a token for a scoped downstream one (RFC 8693)

Tenant claim: signed tenant ID inside the token

Row-level security: database-enforced per-tenant row filter

Break-glass: exceptional, approved, audited staff access

8 Interview questions

Q1. How do you ensure tenant context is never missed? BASIC
Take it from a signed token claim, set it once per request in middleware, enforce it automatically in the data layer with row-level security as a backstop, and propagate it to downstream calls, queues and jobs.

Q2. How should support staff access tenant data? BASIC
Through a separate break-glass role that is approved, time-limited, scoped to one tenant and audited where the customer can see it, never through ordinary tenant roles.

Q3. Why not read the tenant from a request header? BASIC
Anyone can set a header; only a signed token claim, or a server-side membership check, proves which tenant the caller belongs to.

Q4. A user belongs to three tenants. How does the API know which one a request is for? SCENARIO
The client names the tenant, the server checks it against the user's memberships, and ideally issues a short-lived token scoped to that tenant for the rest of the call chain.

Q5. A worker processes queued jobs for all tenants. How do you keep it tenant-safe? SCENARIO
Each message carries its tenant; the worker sets the tenant context (and the RLS setting) before any data access and fails if the tenant is missing.

Q6. Should another tenant's invoice return 403 or 404? TRICKY
404: a 403 confirms the object exists, which leaks information and helps enumeration.

Q7. Are random UUIDs enough to stop invoice-ID guessing? TRICKY
No. They make guessing harder, but IDs leak through links, logs and exports; only a tenant check on every access prevents BOLA.

Q8. RLS uses a session setting. What goes wrong with connection pooling? TRICKY
A plain SET survives on the pooled connection and the next request may inherit another tenant's value; use SET LOCAL inside each transaction.

Next: SD-R-004 · Noisy Neighbour backendroadmap.com